DAY 3 OF 7

REST APIs for the whole app

Every Quiz Master feature working through the API before we write any UI

Today's plan

10 min	Recap: tokens and roles
15 min	REST design: URLs, methods, status codes
40 min	Live code: resources.py (CRUD)
20 min	Live code: attempt, submit, scores
20 min	Live code: users, block, search, demo data
15 min	Test with curl, wrap-up

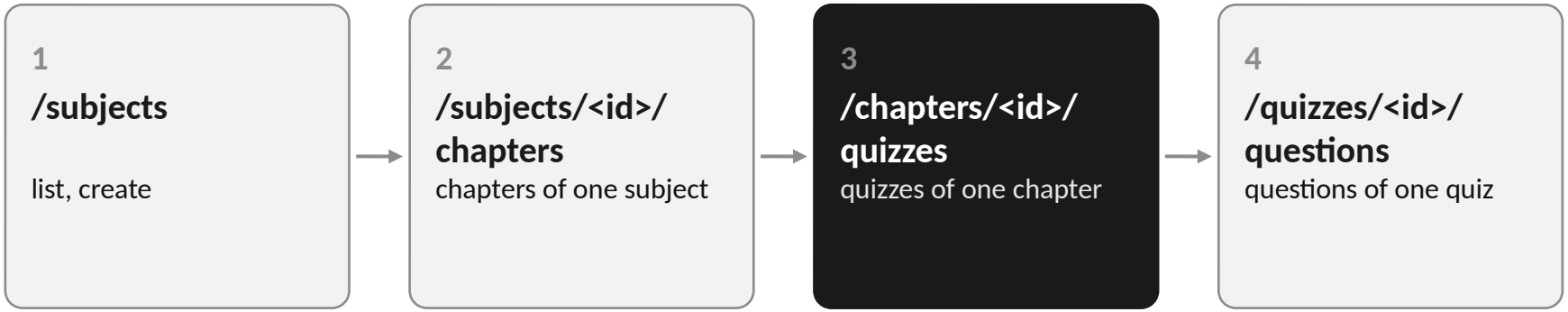
By the end you can

- Design REST URLs for nested resources
- Write CRUD endpoints with validation and correct status codes
- Explain why the score is calculated on the server

REST: URL = thing, method = action

Method	Meaning	Example	Success
`GET`	read	`GET /api/subjects`	200 OK
`POST`	create	`POST /api/subjects`	201 Created
`PUT`	update	`PUT /api/subjects/3`	200 OK
`DELETE`	remove	`DELETE /api/subjects/3`	200 OK

Nested URLs show ownership



Edit and delete use the item's own URL: PUT /api/chapters/7, DELETE /api/questions/12.

Status codes we use



200 / 201

OK / Created



400

Bad input (failed validation)



401 / 403

Not logged in / not allowed



404 / 409

Not found / duplicate

A create endpoint with validation

backend/application/api/resources.py

```
@resources_bp.post("/subjects")
@admin_only
def create_subject():
    data, missing = get_json("name")
    if missing:
        return error("Subject name is required")
    if Subject.query.filter_by(name=data["name"]).first():
        return error("Subject already exists", 409)
    s = Subject(name=data["name"],
                description=data.get("description"))
    db.session.add(s)
    db.session.commit()
    return jsonify(subject_to_dict(s)), 201
```



admin_only

token + admin role in one decorator



Validate first

missing name → 400, duplicate → 409



Serializer

subject_to_dict() keeps JSON consistent

Never trust the client

backend/application/api/user.py

```
# GET /quizzes/<id>/attempt -> questions WITHOUT answers
question_to_dict(q, with_answer=False)

# POST /quizzes/<id>/submit  {"answers": {"5": 2, "6": 1}}
correct = 0
for q in quiz.questions:
    chosen = answers.get(str(q.id))
    if str(chosen) == str(q.correct_option):
        correct += 1

score = Score(quiz_id=quiz.id, user_id=current_user.id,
              total_scored=correct,
              total_questions=len(quiz.questions))
```



Hide answers

correct_option is never sent to a quiz taker



Score on server

the browser only sends the chosen options



Saved

every attempt becomes a Score row

Protect a whole blueprint at once

backend/application/api/admin.py

```
@auth_required("token")
@roles_required("admin")
def _admin_guard():
    return None

@admin_bp.before_request
def protect():
    # CORS pre-flight (OPTIONS) carries no token
    if request.method == "OPTIONS":
        return None
    return _admin_guard()
```

before_request

runs before every route in /api/admin

OPTIONS

let CORS pre-flight through

Search

ilike("%text%") is parameterised – safe from SQL injection

The API at a glance

Method	URL	Who
GET / POST	`/api/subjects`	all / admin
GET / POST	`/api/subjects/<id>/chapters`	all / admin
GET / POST	`/api/chapters/<id>/quizzes`	all / admin
GET / POST	`/api/quizzes/<id>/questions`	admin
GET	`/api/quizzes/<id>/attempt`	user
POST	`/api/quizzes/<id>/submit`	user
GET	`/api/scores`	logged in
GET / PATCH	`/api/admin/users · /users/<id>/toggle`	admin
GET	`/api/admin/search?q=`	admin

Test with curl

```
Terminal

python seed_demo.py     # once
python app.py

TOKEN=<admin token>
curl -X POST localhost:5000/api/subjects \
  -H "Authentication-Token: $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"Maths"}'

curl "localhost:5000/api/admin/search?q=py" \
  -H "Authentication-Token: $TOKEN"
```

Check that...

- Admin can create subject → chapter → quiz → question
- User sees questions without correct_option
- Submit returns the score; /api/scores lists it
- User token on POST /api/subjects → 403

Common errors

Error	Fix
405 Method Not Allowed	wrong HTTP method for that URL
Submit always gives 0	answer keys must be question ids as strings: {"5": 2}
IntegrityError UNIQUE	duplicate name – return 409 before inserting
Deleting a subject fails	missing cascade="all, delete-orphan" in models

Recap

- URLs name resources, methods say what to do, status codes say what happened
- Every write is validated on the server
- Quiz takers never receive the answers; scores are calculated server-side
- The whole admin blueprint is protected in one place

Next: Day 4 – a Vue 3 app with Router and Vuex that logs in with these APIs.